INDEX

ANITS

S.No.	Date	Name of the Experiment	Mapping To Course Outcome	Grade	Signature
1	8/7/26	Introduction to HTML and CSS	CO_1	10	
2	15/7/26	static webpage using HTML and CSS	CO_1	10	
3	15/7/26	HTML codes using audio & video	CO_1	10	
4	22/7/26	program using Javascript	CO_1	10	
5	29/7/26	Dynamic webpage using Javascript	CO_2	10	
6	5/8/26	Implement modules in NodeJs	CO_2	10	
7	12/8/26	Develop miniapplication using Express and NodeJs	CO_2	10	
8.	9/9/26	functional component & class component using react	CO_2	10	
9.	23/9	Implement http method with JSON server	CO_2	10	
10.	30/9	Develop CRUD operation using mongo DB.	CO_3	10	

Anil Neerukonda Institute of Technology & Sciences
(Autonomous)
Affiliated to AU, Approved by AICTE & Accredited by NAAC with A+ Grade
Accredited by NBA (B.Tech – ECE, EEE, CSE, IT, MECH, Civil & Chemical)
Sangivalasa-531 162, Bheemunipatnam Mandal, Visakhapatnam District
Phone: 8712005999, 8712098222, Website: www.anits.edu.in email:

Department of Computer Science & Engineering (AI & ML)

Academic Year:	2026-2027	**Semester:**	1st
Name of the Student:	P. Likhitha	**Roll Number:**	A2126552163
Name of the Lab:	Full-Stack Lab	**Timings:**	9:40 - 12:10

LAB WEEKLY ASSESSMENT SHEET

Date	Experiment /Week No	Observation Book				Total (15 Marks)	Record (10 Marks)	Faculty Remarks
		Theoretical Knowledge (4 Marks)	Programming Knowledge (4Marks)	Results (5Marks)	Viva- Voce (2Marks)			
01/07/26	01	4	4	5	2	15		
08/07/26	02	4	4	5	2	15		
15/07/26	03	4	4	5	1	14		
22/07/26	04	4	4	5	1	15		
29/07/26	05	4	4	5	1	14		
05/08/26	06	4	4	4	2	14		
12/8/26	07	4	4	5	2	15		
19/8/26	08	4	4	4	1	14		
2/9/26	09	4	4	5	2	15		
8/9/26	10	4	4	5	2	15		

Name of the Faculty: Ch. Sruthi

Faculty Signature:

Introduction to HTML and CSS>

→ HTML (Hyper text markup language) is the standard markup language used to create and structure content on webpages. It defines the basic structure of a webpage using elements and tags such as headings, paragraphs, links, images, lists, tables, forms and other multimedia content. HTML documents are interpreted by web browsers to display the content in a structured and meaningful manner. HTML provides the structure and semantic organization of a webpage.

→ CSS (cascading style sheets) is a stylesheet language used to control the presentation and visual appearance of HTML documents. It is used to define properties such as colors, fonts, backgrounds, spacing, borders, alignment, dimensions and layouts. CSS also supports responsive design, allowing webpages to adapt to different screen sizes and devices. CSS can be applied using inline, internal or external.

→ HTML and CSS work together to develop webpages, while CSS provides the structure and content, while CSS provides the styling, layout, visual presentation. The combination of both

forms the foundation of modern web development and enables the creation of organized, attractive and user-friendly web interfaces.

* Difference blw mern stack & full-stack >

→ MERN stack is a Javascript-based technology stack used for developing modern web applications. MERN stands for mongoDB, Express.JS, React.JS, Node.JS. mongoDB is used for database management, Express.js handles the backend framework and APIs React.JS is used to develop frontend user interface and Node.JS provides the server side runtime environment.

→ full-stack development refers to the development of both the frontend and backend of a web application along with database integration. A full stack developer works with technologies used for designing the user interfaces, implementing server-side logic, managing databases.

HTML Components:

1. HTML Tags - Define the structure and elements of a webpages.
2. Headings - used to display titles & headings using <h1> to <h6>
3. paragraphs - used to display text using the <p> tag
4. Links - used to navigate blw webpages using <a> tag
5. Images - used to display images using <img> tag
6. Lists - used to create ordered and unordered lists using <ol>, <ul>,
7. Tables - used to organize data in rows and columns.
8. forms - used to collect user input using elements such as textboxes, buttons and checkboxes.
9. Div and span - used to group and organize webpage content.
10. Semantic elements - Elements such as <header>, <nav>, <section>, <article>, <footer> provide meaningful webpage structure.

1. Selectors - Select HTML elements to apply styles
2. Properties - Defines the visual characteristic of elements
3. Values - Specify the setting of a css property
4. Colors - Define text, background, border colors.
5. fonts - control font family, size, style and weight.
6. Backgrounds - Set background colors and image.
7. Borders - Define the border style, width and radius.
8. margin - controls the space outside an element
9. padding - controls the arrangement of elements
10. layout - controls the arrangement of elements using flexbox, Grid and positioning.

HTML	HTML5
→ older version of HTML	→ Latest major version HTML supports audio and video.
→ Limited multimedia support	→ No external plugs needed.
→ Uses external plugins for multimedia	→ Provides semantic tags
→ Has fewer semantic tags	Support <canvas>
→ No <canvas> element	Support local storage
→ Limited web storage	simple <!DOCTYPE HTML>
→ complex DOCTYPE declaration.	

CSS vs CSS3

CSS	CSS3
→ Earlier version of CSS Basic styling features	→ Advanced version of CSS provides advanced styling features
→ Limited Animations	→ supports animation & transition.
→ No flexible layout systems limited visual effects.	→ supports flexbox & grid.
→ No media queries	→ supports shadows, gradients effects
uses a single specification	→ supports media queries

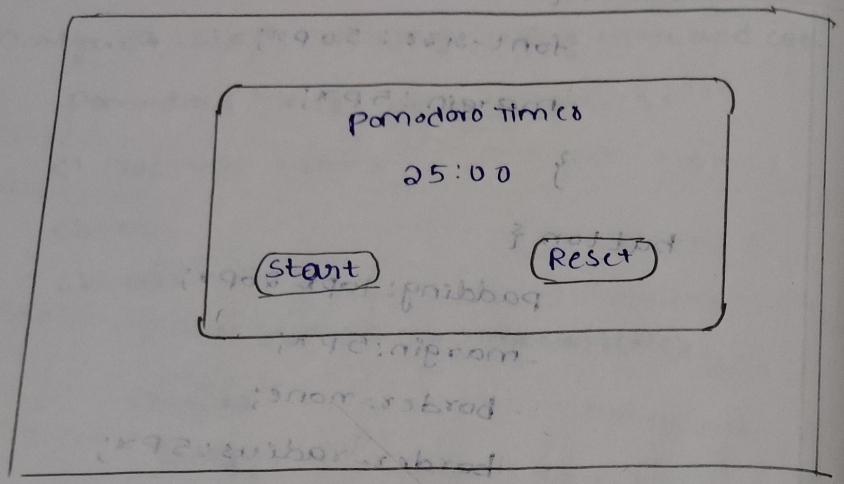
15/07/26

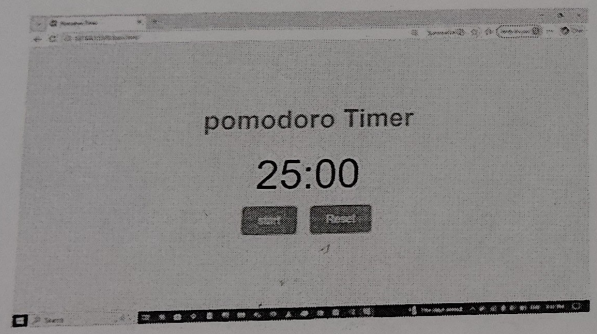
1- Design a static webpage using HTML and CSS.

Pomodoro Timer, using HTML, CSS.

```
<!DOCTYPE html>
<html>
<head>
    <title> Pomodoro Timer </title>
    <style>
    body {
        font-family: Arial;
        background: #f5f5f5;
        txt-align: center;
        padding: 50 px;
    }

.timer {
    width: 300px;
    margin: auto;
    padding: 30px;
    background: white;
    border-radius: 15px;
    box-shadow: 0 0 10px gray;
}

h1 {
```

```
.time {
    font-size: 50px;
    margin: 5px;
}

button {
    padding: 10px 20px;
    margin: 5px;
    border: none;
    border-radius: 5px;
    background: #e74c3c;
    color: white;
}

</style>
</head>
<body>
    <div class = "Timer">
    <h1> Promodoro Timer </h1>
    <div class = "time"> 25:00 </div>
    <button> Start </button>
    <button> Reset </button>
    </div>
</body>
</html>
```





22/07/2026

(2)

(i) Design an HTML code using audio tags.

```
<! DOCTYPE html>
<html>
 <head>
    <title> Audio Example </title>
 </head>
 <body>
      <h2> My favourite song </h2>
      <audio controls>
       <source src = "song.mp3" type =
                          "audio/mpeg";>
       your browser does not support
       the audio tag </audio>
```

o/p: Displays an audio player with play, pause and volume controls.

Steps:

Create a new HTML file

Add the basic HTML structure

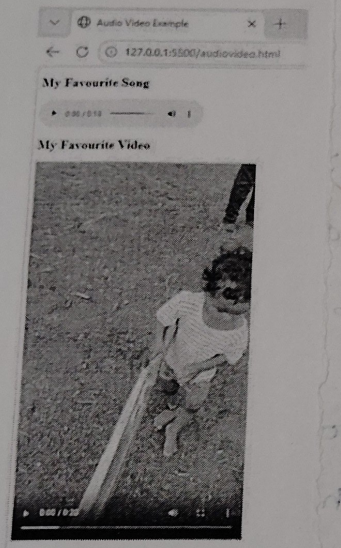
Add heading using <h2> tag

Insert the <audio> tag with controls attribute

Specify the audio file using <source> tag.

Save and open the HTML file in a web browser.

Play the audio using displayed controls.



(ii) Design a HTML code using video tags.

```
<! DOCTYPE html>
<html>
  <head>
    <title> video example </title>
  </head>
<body>
    <h2> My video</h2>
    <video width = "400" controls>
      <source src: "video.mpy"
        type : "video/mpy">
    </video>
</body>
</html>
```

Steps:

Create a new HTML file.

Add a basic HTML structure.

Add a heading using the <h2> tag.

Insert the <video> tag with controls attribute

Specify the video file using <source> tag.

Save the file and open it in web browser.

3. Write a program using javascript (arrays, function).

```
let numbers = [12, 45, 8, 23, 56, 19, 34, 67,
               90, 11];

function displayArray(arr) {
    console.log("Array Elements:");
    for (let i=0; i<arr.length; i++){
        console.log(arr[i]);
    }
}
function findsum(arr){
    let sum=0;
    for (let i=0; i<arr.length; i++){
        sum+=arr[i];
    }
    return sum;
}
```

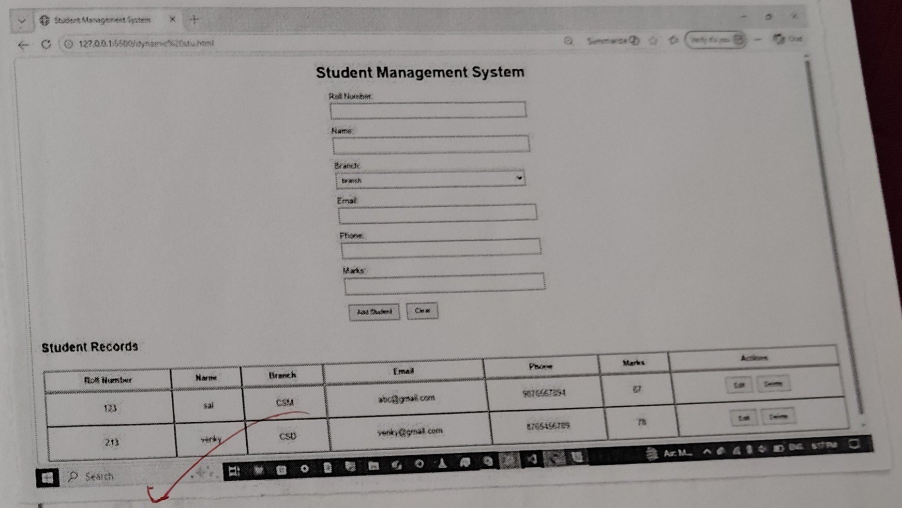
4. Develop Dynamic webpage using Javascript

```
<!DOCTYPE html>
<html>
<head>
    <title>student management system</title>
    <style>
        body {
            font-family: Arial;
            marin : 20px;
        }
        h1{
            text_align:center;
        }
        form {
            width : 400px;
            margin : auto;
        }
        input select {
            width : 100px;
            padding: 8px;
            margin: 5px 015px;
        }
        button {
            padding: 8px 15px;
            margin: 5px;
        }
```

```
table {
    width: 100%;
    margin-top: 30px;
}
th, td {
    border: 1px solid black;
    padding: 10px;
    text-align: center;
}
</style>
</head>
<body>
<h1>Student management system</h1>
<form id="studentform">
    <label>Roll Number:</label>
    <input type="text" id="roll" required>
    <label>Name:</label>
    <input type="text" id="name" required>
    <label>Branch:</label>
    <select id="branch">
        <option>branch</option>
        <option>CSE</option>
        <option>CSM</option>
        <option>CSD</option>
    </select>
    <label>Email:</label>
    <input type="email" id="email" required>
    <label>phone:</label>
    <input type="tel" id="phone" required>
    <label>marks:</label>
    <input type="number" id="marks" required>
    <button type="submit">Add student</button>
    <button type="button" onclick="clearform()">
    </button>
</form>
<h2>Student Records</h2>
<table>
    <thead>
        <tr>
            <th>Roll Number</th>
            <th>Name</th>
            <th>Branch</th>
            <th>Email</th>
            <th>phone</th>
            <th>marks</th>
            <th>Actions</th>
        </tr>
    </thead>
    <tbody id="studenttable">
    </tbody>
```

```
let students = [];
let editIndex = -1;
document.getElementById("studentform").
    addEventListener("submit", function(event){
event.preventDefault();
let roll = document.getElementById("roll").value;
let name = document.getElementById("name").value;
let branch = document.getElementById("Branch").
                value;
let email = document.getElement ById("email").
                value;
let phone = document.getElementById("phone").
                value;
let marks = document.getElementById("marks").
                value;
let student = {
    roll: roll,
    name: name,
    branch: branch,
    email: email,
    phone: phone,
    marks: marks };
if (editIndex == -1) {
    students.push(student);
} else {
    editIndex = -1;
}
displayStudents();
clearForm();
});
function displayStudents() {
    let table = document.getElementById
                ("studentTable");
    table.innerHTML = "";
for (let i=0; i<students.length; i++) {
    let row = "<tr>";
    row += "<td>" + students[i].roll + "</td>";
    row += "<td>" + students[i].name + "</td>";
    row += "<td>" + students[i].branch + "</td>";
    row += "<td>" + student[i].email + "</td>";
    row += "<td>" + student[i].phone + "</td>";
    row += "<td>" + student[i].marks + "</td>";
    row += "<td>";
    row += "<button onclick='editStudent("+i+")'>
            Edit</button>";
row += "<button onclick='DeleteStudent("+i+")'>
            Delete</button>";
    row += "</td>";
    row += "</tr>";
```

```
}
}
function editstudent(index){
    document.getElementById("roll").value
        = students[index].roll;
    document.getElementById("name").value
        = students[index].name;
    document.getElementById("branch").value
        = students[index].branch;
    document.getElementById("email").value
        = students[index].email;
    document.getElementById("phone").value
        = students[index].phone;
    document.getElementById("marks").value
        = students[index].marks;
    editIndex = index;
}
function deletestudent(index){
    students.splice(index,1);
    displaystudents();
}
function clearform(){
    document.getElementById("studentform")
        .reset();
    editIndex = -1;
}
</script>
</body>
</html>
```



5. Implement modules in NodeJS

folder structure:

```
Node modules/
    app.js
    math.js
    package.json
```

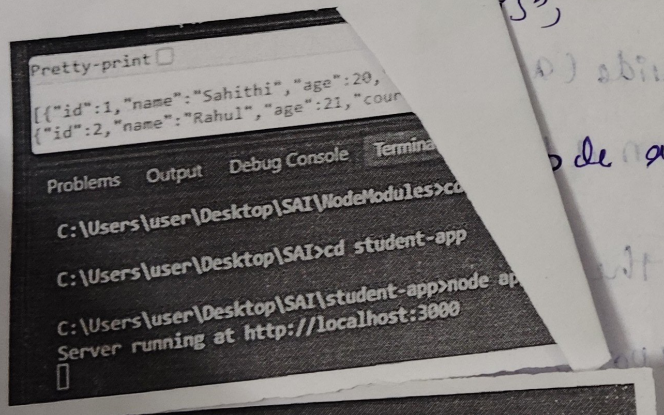
math.js

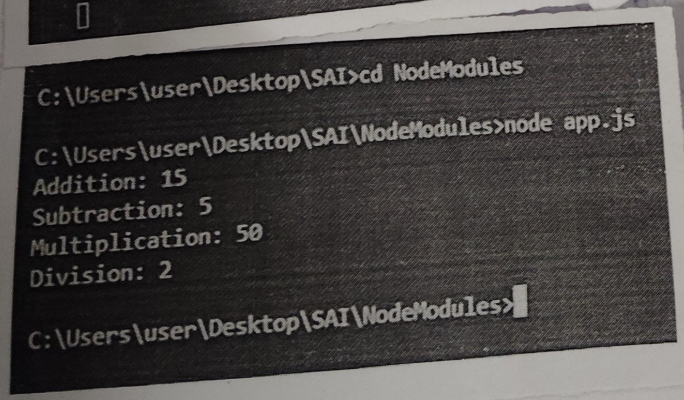
```
//creating a module
function add(a,b){
    return a+b;
}
function subtract(a,b){
    return a-b;
}
function multiply(a,b){
    return a*b;
}
function divide(a,b){
    return a/b;
}
//Exporting the functions.
module.exports={
    add,
    subtract,
    multiply,
    divide
};
```

```
Const math = require (
console.log ("Addition:", math.add(10,5));
console.log ("subtraction:", math.subtract
                              (10,5));
console.log ("multiplication:", math.multiply
                              (10,5));
console.log ("Division:", math.divide(10,5));
```

package.json:

```
{
  "name": "node_modules",
  "version": "1.0.0",
  "description": "sample Node.js module
                  xample;
                  ";
                  de app.js
```





6- Develop mini application using express and Nodejs.

folder structure:

```
Student-app/
        node_modules/
        app.js
        index.html
        package_lock.json
        package.json
```

→ mkdir student-app
→ cd student-app
→ npm init -y
→ npm install express.

app.js:

```
const express = require("express");
const app = express();
const PORT = 3000;
app.use (express.json());
let students = [
    { id: 1,
      name: "sai",
      age: 19,
      course: "B.Tech"
    },
    {id: 2,
     name: "Rahul",
     age: 21,
     course: "B.Tech"
     }
];
```

```
app.get("/", (req,res) => {
    res.sendfile(__dirname+"/index.html");
});
app.get("/students", (req,res) => {
    res.json(students);
});
app.get("/students/:id", (req,res) => {
    const id = parseInt(req.params.id);
    const student = students.find(s=>s.id===id);
    if(student){
        res.json(student);
    }else{
        res.status(404).json({
            message: "student not found"
        });
    }
});
app.post("/students", (req,res) => {
    const newStudent = {
        id: students.length+1;
        name: req.body.name;
        age: req.body.age;
        course: req.body.course;
    };
    students.push(newStudent);
    res.status(201).json({
        message: "student added
        student: newStudent
    });
});
app.delete("/students/:id", (req,res) => {
    const id = parseInt(req.params.id);
    const index = students.findIndex(s=>s.id===id);
    if(index!==-1){
        student.splice(index,1);
        res.json({
            message: "student deleted successfully"
        });
    }else{
        yes.status(404).json({
            message: "student not found"
        });
    }
});
app.listen(PORT,()=>{
    console.log("server running at
    http://localhost:${PORT}");
});
```

index.html;

```
<!DOCTYPE html>
<html>
<head>
  <title>Student management system</title>
```

```
<body>
    <h1> student management system </h1>
    <h2> Welcome! </h2>
    <p> This is a simple Node.js and
        Express.js application. </p>
    <h3> Available operations </h3>
    <ul>
        <li><a href="/students"> view All
                students </a> </li>
        <li> use/ students/ : to view student
            </li>
    </ul>
</body>
</html>.
```

07 Create a functional component and class component using react.

Aim: To develop a react application using functional components and class components to display student details, manage state, handle button clicks and dynamically update data on a webpage.

- Functional component:

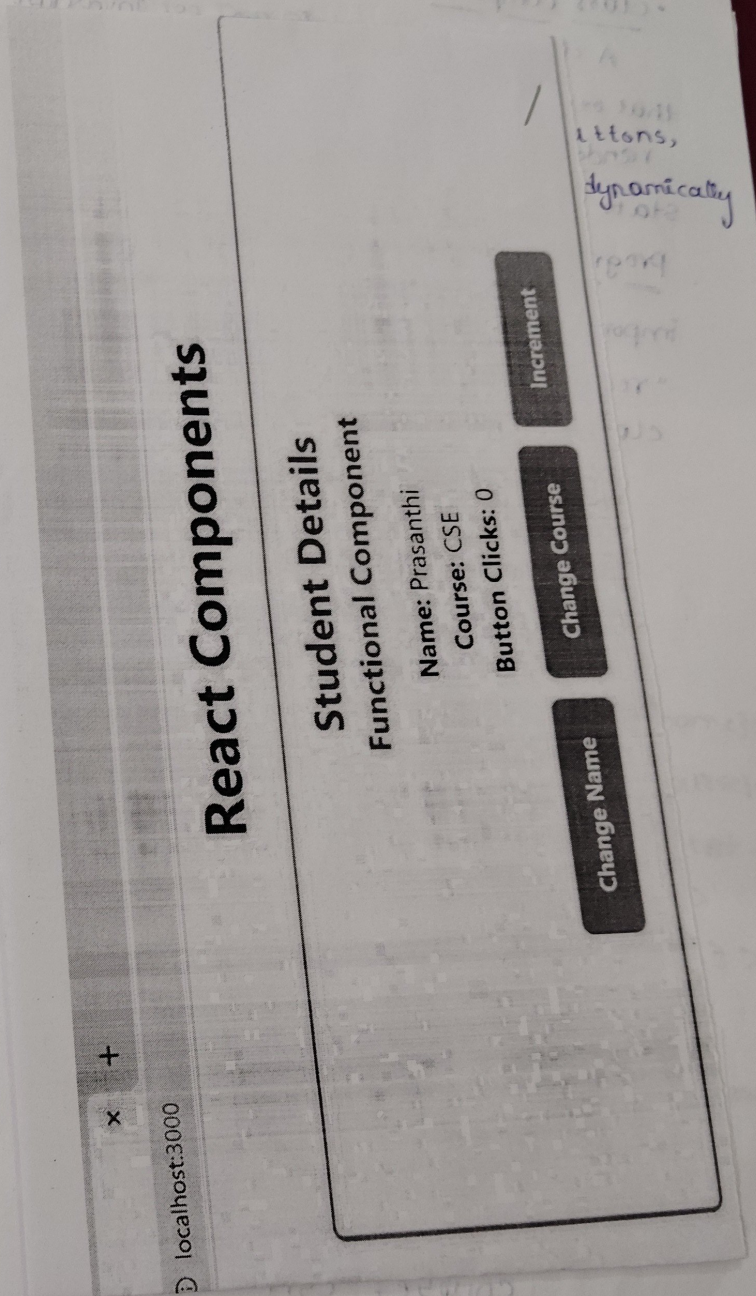
A functional component is a javascript function that returns JSX to display the user interface. It uses React Hooks such as useState to manage state and update the webpage dynamically.

Program:

```
import React, {useState} from
'react';
function studentfunctional() {
    Const [name, setName] = useState("Likhitha");
    Const [course, setCourse] = usestate("CSM");
    Const [count, setCount] = usestate(0);
    Const changeName = () => {
        SetName("Likhi");
    };
```

```
    setCourse ("CSD");
  };
const incrementCount = () => {
    setCount (count + 1)
  };
return (
 <div style = {{
    border : "2px Solid blue",
    padding : "20px",
    margin : "20px",
    borderRadius : "10px",
    textAlign : "center"
 }} >
 <h1> Student Details </h1>
 <h2> Functional Component </h2>
 <p> <b> Name: </b> {name} </p>
 <p> <b> Course: </b> {course} </p>
 <p> <b> Button Clicks: </b> {count} </p>
 <button onClick = {changeName} >
      ChangeName </button>
<button onClick = {changecourse} change
  Course </button>
```

```
<button onClick = {incrementCount} > Increment
</button>
```



buttons, dynamically

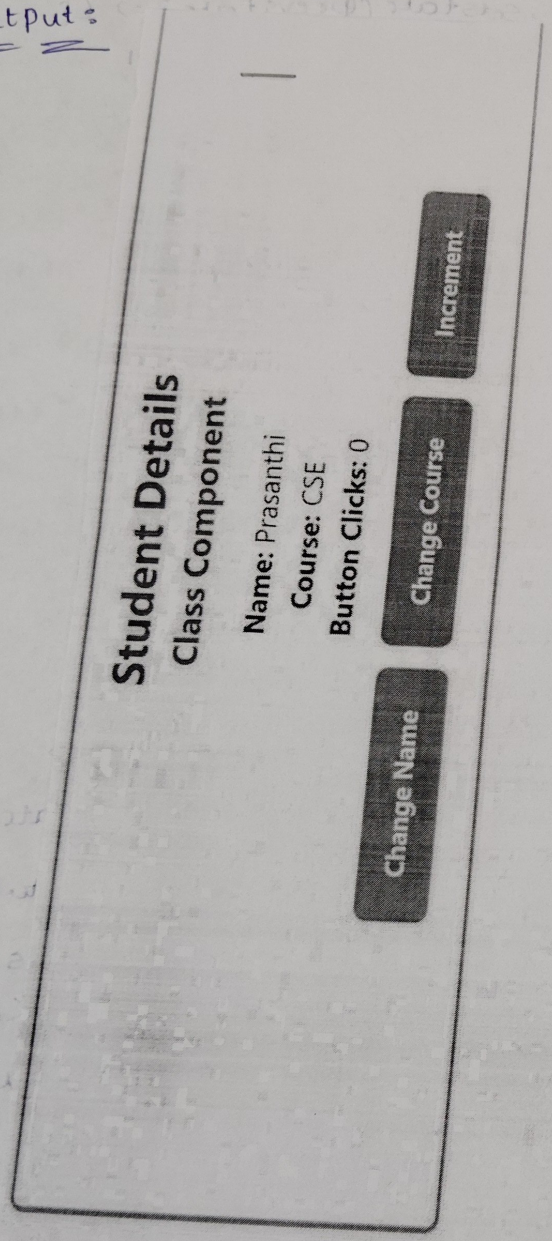
• Class component:

A class component is an ES6 Javascript class that extends React component. It uses the render() method to return JSX and manages state using this.state and this.setState().

Program:

```
import React, {Component} from
"react";
class StudentClass extends Component {
    constructor(props){
        super(props);
        this.state = {
            name: "Likhitha",
            course: "computer science",
            count: 0
        };
    }
    changeName = () => {
        this.setState({
            name: "Likhi"
        });
    };
    changeCourse = () => {
        this.setState({
            course: "CSM"
        });
    };
    incrementCount = () => {
        this.setState((prevState) => ({
            count: prevState.count+1
        }));
    };
    render() {
    return (
        <div style={{
            border: "2px solid green",
            padding: "20px",
            margin: "20px",
            borderRadius: "10px",
            textAlign: "center"
        }}>
            <h1> Student Details </h1>
            <h2> class Component </h2>
            <p><b> Name: </b> {this.state.name}</p>
            <p> <b> course: </b> {this.state.course}</p>
            <p> <b> Button clicks: </b> {this.state.
                count} </p>
            <button onClick={this.changeName}>
                Change Name </button>
            <button onClick={this.changeCourse}>
                Change Course </button>
            <button onClick={this.incrementCount}>
                Increement </button>
        </div>
```

Output:



6. Implement http methods using JSON Server

Aim: To implement http methods such as GET, POST, PUT, PATCH, and DELETE using JSON server to perform CRUD (create, Read, Update and Delete) operations on student records.

Theory: JSON Server is a tool used to create a fake REST API using a JSON file. It allows us to perform HTTP operations without developing a backend server from scratch. HTTP methods are used to communicate blw a client and a server.

- GET - Retrieves data from the server
- POST - Adds new data to the server
- PUT - Replaces an existing record
- PATCH - Updates selected fields of a record
- DELETE - Removes a record from the server.

a. create a JSON server

Step 1: Initialize the project

Open the VS Code terminal and execute the following commands:

```
mkdir JSONServer
cd JSONServer
npm init -y
npm install json server
```

Step2: Create db.json

Create a file named db.json and add the following student data

```
{
  "students": [
    { "id": "1",
      "name": "Likhitha"
      "course": "CSM",
      "marks": 90
    },
    { "id": "2"
      "name": "Sravani"
      "course": "IT",
      "marks": 85
    }
```

Step3: Start the JSON Server

for the current JSON Server version,

Execute:

```
npx.json.server db.json;
```

The Server start at:

http://localhost:3000

The student records can be accessed using

http://localhost:3000/students

2. Implementation of HTTP methods

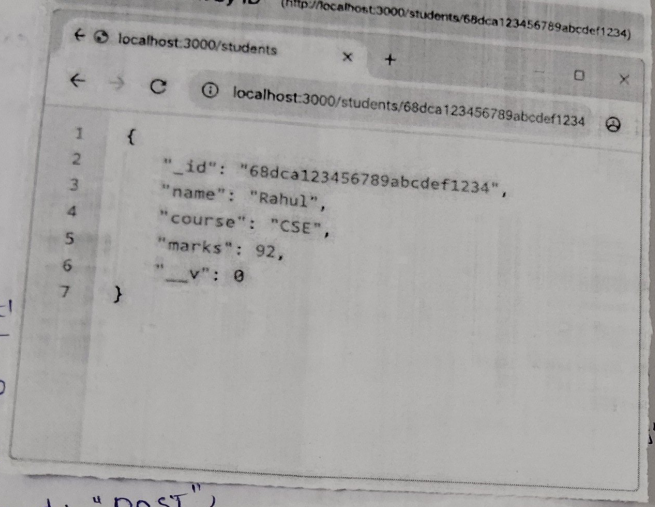
a. GET method: The GET method is used to retrieve student records from json server

Code:

```
fetch("http://localhost:3000/students")
.then(response => response.json())
.then(data => {
    Console.log(data);
})
.catch(error => {
    Console.log(error);
});
```

output:

2. GET Student by ID (http://localhost:3000/students/68dca123456789abcdef1234)



B. post meth

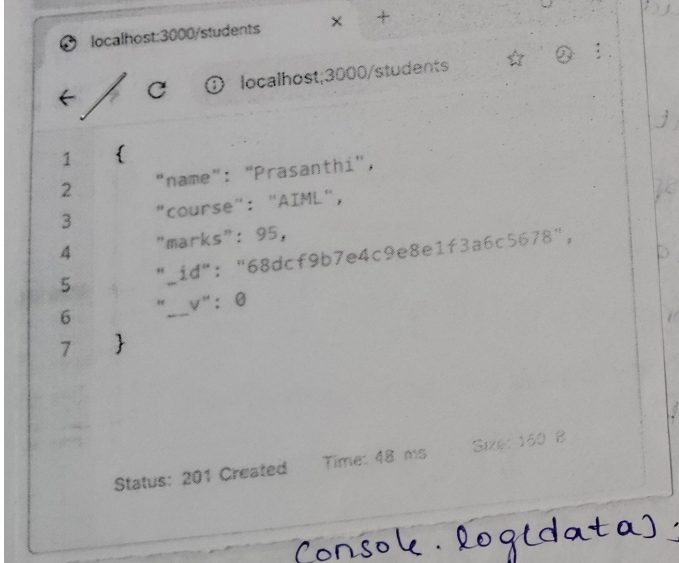
insert o

code: fet

method: "POST")

cation lison";

3. POST (Create Student) – Using Postman



.json())

console.log(data);

});

.catch(error => {

console.log(error);

});

});

output:

c. put method: The put method is used to replace an existing student record with updated data

code: fetch("http://localhost:3000/students/",{

method: "PUT",

headers: {

"content-type": "application/json"

},

body: JSON.stringify({

name: "Tejaswari",

course: "ATML",

marks: 95

})

})

.then(response => response.json())

.then(data => {

console.log(data);

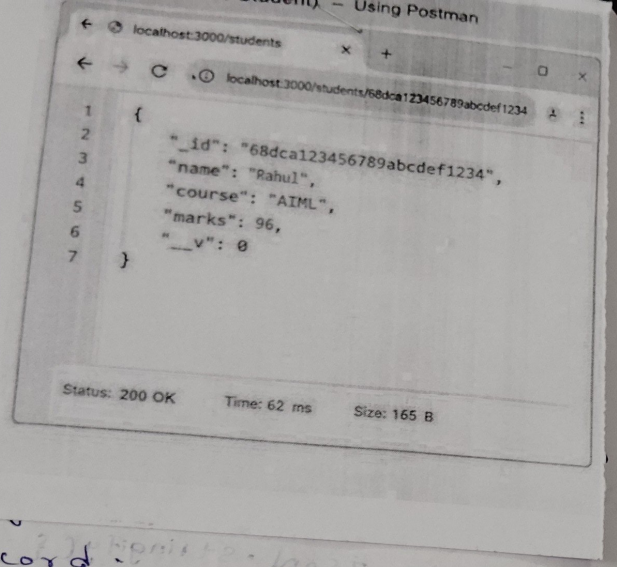
})

.catch(error => {

console.log(error);

});

output:



D- Patch r.

Updatc

Student record :

Code: fetch ("http://localhost:3000/students/

2", {

method: "PATCH",

headers: {

"content-type": "application/jso"

},

body: JSON.stringify({

marks: 96

})

})

.then (response => response.json())

.then (data => {

Console.log(data);

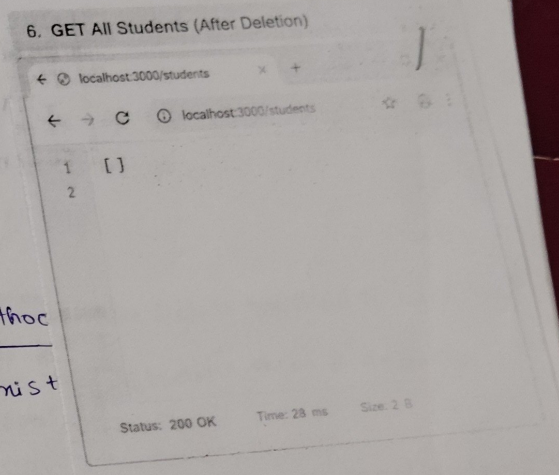
})

.catch(error => {

Console.log(error);

});

output:



E. Delete methoc

remove one exist

server.

code: fetch ("ht

method: "DELETE"

})

.then (response => {

if (response.ok) {

console.log ("student deleted

successfully");

}

})

.catch (error => {

console.log (error);

});

outpu

Student) – Using Postman

localhost:3000/students

localhost:3000/students/68dca123456789abcdef1234

```
{
    "message": "Student deleted successfully"
}
```

Status: 200 OK Time: 45 ms Size: 46 B

. Af

and DELETE

student will

C

, PUT, PATCH,

ly implemented

perations were

the data was

d delete d

1. GET All Students (http://localhost:3000/students)

localhost:3000/students

localhost:3000/students

```
[
  {
    "_id": "68dca123456789abcdef1234",
    "name": "Rahul",
    "course": "CSE",
    "marks": 92,
    "__v": 0
  }
]
```

Experiment-09:

9) Develop CRUD operations using MongoDB

Theory: MongoDB is a Nosql database that stores data in flexible, JSON like documents, CRUD-operations are used to manage data in database.

MongoDB is an object Data modelling (ODM) library for mongoDB and Node.js it helps define schemes, interact with Collections and perform database operations.

1. Installation and project setup:

```
mkdir mongoDB-CRUD
cd mongoDB-CRUB
npm init-y
npm install express mongoose
```

Program: Server.js

```
Const express = require("express");
Const mongoose = require("mongoose");
Const app = express();
Const port = 3000;
app.use(express.json());
```

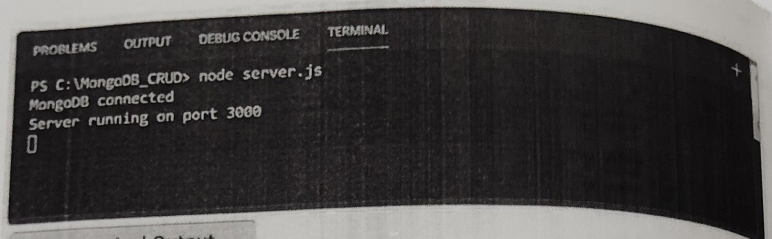
```
mongoose.connect("mongodb://127.0.0.1:27017/college")
.then() → console.log("mongoDB Connected"))
.catch(err → console.log(err));

// Create Student
const studentSchema = new mongoose.Schema({
    name: {
        type: String,
        required: true
    },
    Course: {
        type: String,
        required: true
    },
    marks: {
        type: Number
        required: true
    }
});

// create Student model
const Student = mongoose.model("Student", studentSchema);
```

```
// Create : Add
app.post('/students', async (req, res) => {
  try {
    const student = new Student(req.body);
    const savedStudent = await student.save();
    res.status(201).json(savedStudent);
  } catch (error) {
    res.status(400).json({ message: { error: message } });
  }
});

// Read: Get all students
app.get("/students", async (req, res) => {
  try {
    const students = await Student.find();
    res.status(500).json({ message: error: message });
  }
});

// Read: Get student by ID
```

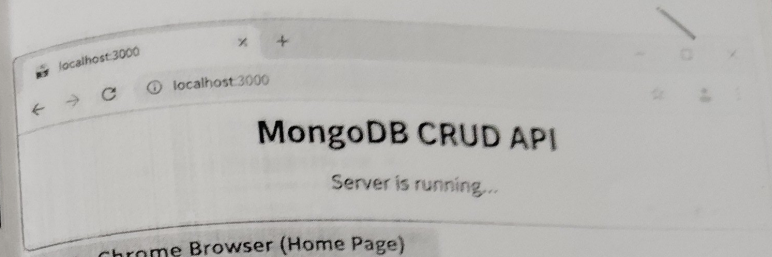
```
app.get("/students/:id", async (req, res) =>{
  try{
    const student = await
      Student.findById(req.params.id);
  if (!student){
    return res.status(404).json({
    message:"student not found"
  });
}
res.status(200).json(student)
} catch(error){
  res.status(400).json({message:
        error.message});
  }
});
//update
app.put("/students/:id", async
  (req,res) =>{
try{
  const student = await
```

```
Student.findByIdAndUpdate(
  req.params.id,
  req.body,
  {new:true, runValidators:true}
  );
if(!student){
  return res.status(404).json({
    message:"student not found"
  });
}
res.status(200).json(student);
} catch(error){
  res.status(400).json({message:
        error.message});
  }
});
// Delete
app.delete("/students/:id", async
  (requires) =>{
```

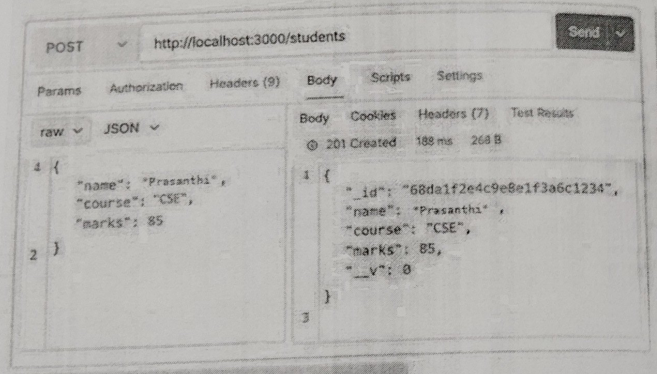
```
try {
const student = await Student.
    findByIdAndDelete(
req.params.id);
if(!student){
   return res.status(404).
       json({
    message: "student not found"
});

}
res.status(200).json({
  message: "Student deleted
    successfully" });
} catch (error) {
   res.status(400).json
      ({message:
        error.message});
}

});
```



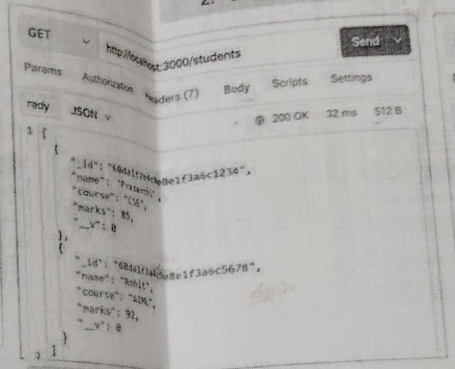
1. Terminal Output



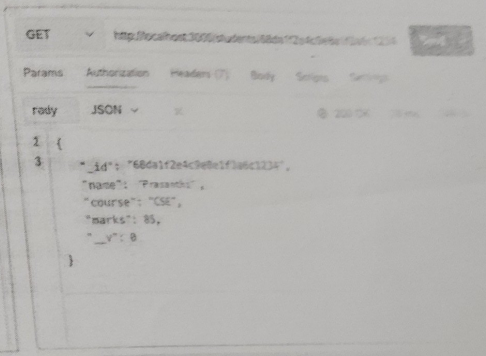
2. Chrome Browser (Home Page)



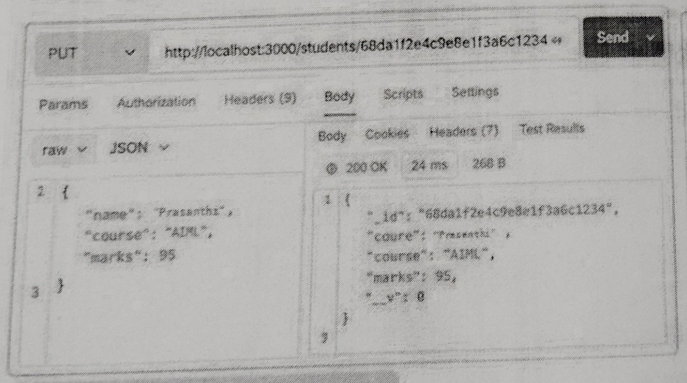
3. Create (POST) – Add Student



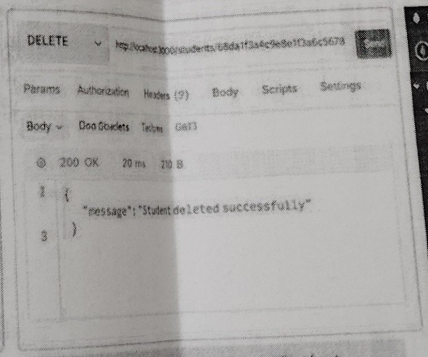
4. Read (GET) – Get All Students



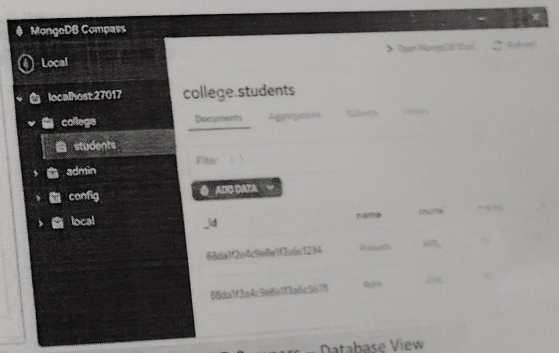
5. Read (GET) – Get Student by ID



6. Update (PUT) – Update Student



7. Delete (DELETE) – Remove Student



8. MongoDB Compass – Database View